Dynamic Large Language Model Quizzing for Enhanced Language Acquisition in Augmented Reality

Category: Research

Paper Type: algorithm/technique and evaluation

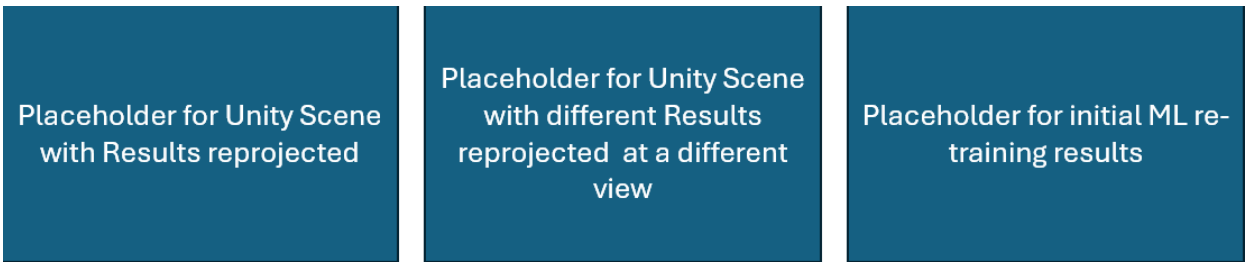


Figure 1: Examples of our language learning framework including in-situ labels reprojected into the scene from different views (left and middle), and a diagram of our initial machine learning results that show how our interactive re-training approach can improve context-specific recognition.

ABSTRACT

In-situ object labeling has been shown to lead to better vocabulary retention for new language acquisition. However, many of the current approaches are limited to showing an object label superimposed over a recognized object. In this paper, we present a system by which a user can dynamically receive instruction on or testing of foreign language vocabulary and concepts using Large Language Models(LLM). This results in more significant language-learning events, leading to longer retention of the concepts.

Keywords: Augmented Reality, Language Learning, Artificial Intelligence, Computer Vision

1 INTRODUCTION AND PRIOR WORK (BRANDON AND ERKIN, RESPECTIVELY)

The goal of our project is to improve the efficiency in learning languages using augmented reality and machine learning. A brief description of our system process: get feed from a live feed (webcam), send those images to our machine learning model, send the labels and position of the detection back to the live feed where user can see labels. Through testing and recording trends, we have observed the effects of training and retraining a model. [**?**].

2 RELATED WORK

Perry discussed a quest-based language-learning game using augmented reality. [5]

van Doormaal et al. discusses a method to track object locations in a 3d environment, and save this location for future use. https://www.ncbi.nlm.nih.gov/pmc/articles/PMC6995446/ [7]

Wu et al. studied a robot designed to make learning English more interesting to Chinese children to enhance their language acquisition process. [8]

Parmaxi et al. did a review of current augmented reality methods and technologies for language acquisition, showing that the methodologies can be effective but need more study done on user experience and ease of instruction. [4]

Kanayama et al. discusses the effect that doing poorly on a foreign language vocabulary quiz has on long-term vocabulary retention. [3]

Rowland discusses the effects that language testing has on vocabulary retention, and compares it to the efficacy of restudying concepts. [6]

Weerasinghe et al. discusses the effects that AR supported by Keyword Visualizations has on Learning Vocabulary in a foreign language. [1]

Ibrahim et al. discusses the effects that AR has on Vocabulary Learning. []

3 SYSTEM DESIGN

3.1 Environment

3.1.1 Hardware

The project utilizes a computer with an (Need computer specs)AMD Ryzen 7 5800X @ 3.80 Ghz with a NVIDIA GeForce RTX 3060 Ti graphics card, an HTC Vive VR headset, and a webcamera.

3.1.2 Software

The project uses the following software:

Unity Editor 2021.3.25f1: The Unity game development engine's specific version used for creating the multi-user environment and managing assets, scenes, and scripts.

Yolov8:

Python (version):

SteamVR:

By utilizing these software tools together, the project achieves a an immersive environment, and allows for computer vision recognitions.

3.2 Unity Scene Setup

Within the system's scene in Unity, there are several major components. There is a camera rig object, representing the VR headset and location and positioning, a quad object housing a webcam texture, an object to serve as a file reader and processor, and another object holding the script to handle the projection of lines through the quad to discern where the real-world objects are calculated to be.

The camera rig object contains several SteamVR scripts to handle the positioning, movement, and facing of the "player" throughout the scene. As the player moves through the scene, the camera rig will track their movements and allow data to be sent to the computer vision model with information such as timestamp and location at time of recognition.

Attached to the camera rig is a quad object. This quad holds a script to enable a video feed from a webcam attached to the front of the Vive, and save images from the video feed to be sent through the computer vision model and obtain recognition data. The quad needs to be attached to the camera rig so that it can follow the front of the camera, and move with the rig, allowing a pseudo-video-passthrough. Separately, there is a file processor that reads data received from the computer vision script and processes it, allowing it to be brought back into the scene.

There is an object holding a script that processes all data received from the computer vision model, along with the metadata that has been tracked across both programs, and collects, displays, and aggregates recognized object tags back into the Unity scene

The initial step of replicating the computer lab within Unity and Blender was to thoroughly measure important aspects of the lab. The objects include desks, walls, floors, and other large static objects and equipment. From these measurements, Unity and Blender was used to digitally replicate the environment.

Blender was used to design finer details of the lab's equipment. Specifically objects that are not easily created with Unity's 3d Object selection. In Blender, irregular objects can be customized and designed to scale and easily imported into the Unity scene.

In Unity, spatial layout of the lab was important. Meticulously aligning, virtual walls, desks, and equipment to mirror the physical space. All objects are then combined together in the Unity scene in a one to one scale. All objects must be accurately represented to allow for accurate testing. Testing the Unity space in virtual reality is important for making adjustments to perfect the layout of the environment.

This virtual environment is necessary for testing of the quizzing and other subsequent features. Virtual reality is also easier to test versus an Augmented Reality system. But this complete system should be convertible into an Augmented Reality application. When the user enters the system, the objects in virtual environment should be accurately depict their environment.

Objects recognized with the Yolov8 system will be assigned a prefab. A prefab is short for a 'prefabricated object' and it is a reusable asset complete with its own associated properties, scripts, and behaviors and can be instantiated multiple times throughout the application. The prefab contains multiple elements that will aid the user in learning foreign words. The prefab includes a in-situ text label that can be converted from the player's native language to their target foreign language.

In order to generate words into their corresponding foreign language a dictionary can be written into a script. Words from a CSV (comma-separated values) file can be used in this dictionary. The CSV file will be set up so that the native words are placed in the first column (key) and the foreign words are placed in the second column(value).



Figure 2: A visualization of the Unity Scene Setup

3.3 Learning System

The learning component of the system draws from Anki learning system, which operates on the foundational principles of spaced repetition. Anki's methodology revolves around strategically timed review sessions aimed at optimizing the long-term retention of information. To adapt this system into a different learning environment, we propose a method where users are presented with a word displayed above an object. By clicking on the displayed word, users can toggle it to reveal the corresponding foreign word, fostering an interactive and immersive learning experience. The displaying of words above objects has taken inspiration from work done in the ARbis Pictus paper [2]. In the ARbis Pictus system, corresponding foreign words are placed above the objects with a audio recording of that object.

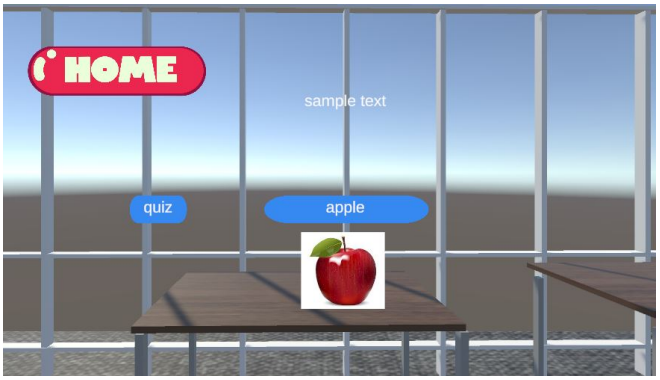


Figure 3: A visualization of the text label above the simulated object

Following this interaction, users are prompted with three distinct buttons, each representing varying degrees of understanding: "Don't Know", "Recognized", and "Know". These buttons emulate the feedback mechanism in the Anki system, allowing users to assess their comprehension level of the presented material. Depending on the button clicked, the system dynamically adjusts the visibility of the object's label, temporarily hiding it for a specified duration before reintroducing it into the learning environment. Notably, the duration of concealment increases incrementally as users progress from indicating unfamiliarity, "Don't Know", to familiarity ,"Know".



Figure 4: A visualization of the language translation and the knowledge rating buttons. These are presented when the text label button is pressed (see Figure 2)

3.4 Quizzing System

Within the learning system, an additional feature enriching the user experience is the incorporation of a multiple-choice quiz. This interactive assessment tool serves as a means for users to gauge their understanding against a broader spectrum of vocabulary words. The decision to implement a multiple-choice format stems from its simplicity and effectiveness. Drawing upon the data stored in the CSV file, the quiz dynamically selects words that share the same initial letter as the target word. This strategic selection not only enriches the learning experience but also introduces an element of challenge. The versatility of this quiz can extend beyond the vocabulary assessment. It can be adapted for practical applications such as fill-in-the-blank exercises. By presenting users with a selection of answer choices, each representing potential completions for a given sentence or phrase, the

quiz becomes a versatile tool for reinforcing contextual understanding and language proficiency. The multiple choice quiz can be combined with the Anki learning system as a way to either increase or decrease the time between hiding and showing words.

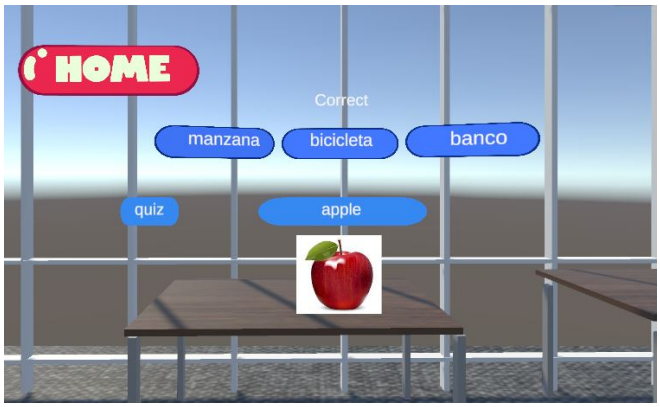


Figure 5: A visualization of the quizzing system. Result is located above the quiz

3.5 Reprojection Algorithm (Brandon)

Obtaining accurate projections of object tags in Unity space was something of a challenge. Because of the continuous movement expected while using the program and the expected delay between saving an image, processing it, and reprojecting the tag, an algorithm was developed to allow the system to more accurately place objects in Unity space.

First, a naming convention was set up containing a timestamp for each photo, along with the position which the photo was taking from, and the direction the user was facing. This metadata was persistent throughout all processes, so that if there were unexpected delays at any step, an accurate reprojection was able to be made. All photos were stored in a folder, which was then processed by the computer vision model's handling script. This script would then return a text file, being named the same as the photo, and containing all the data for any recognitions that were made on objects in the experiment space. This information was processed by the file processing handler in Unity, reading both the metadata stored in the file name and any data from recognitions made by the computer vision model, and all data is sent to the projection script. This script uses the location data to calculate where the quad would have been upon taking the photo, along with the location of each object on the quad, and casts a ray from the historical location of the camera rig to the calculated position of the recognized object, and through the quad until it collides with something in the scene, where a tag is placed.

3.6 Point Aggregation Algorithm (Tucker)

The system tracks objects by storing projections of the objects location. However, these projections can not be shown directly to the user. If labels and quizzes appear all over the view of the user every frame, the user would quickly become overwhelmed. The Image processing model is not always correct or accurate, leading to erroneous projections. The purpose of the Point Aggregation Algorithm is to transform the intermediate data of each projection into points confidently displayed to the user. The goal of creating this algorithm is to show the user a single identification of an object in an accurate position relative to the head set and distinguish it from other projections of items near it.

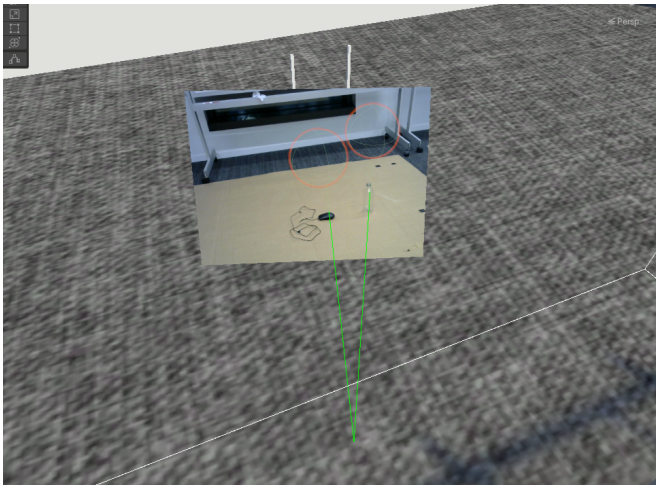


Figure 6: A bottle and a wired mouse being identified by the system.

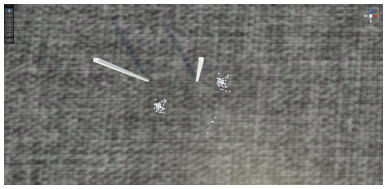
The implementation of this algorithm has three major components: an object projection as input, a prediction controller which runs the algorithm, and an aggregated prediction as an output. The object projection holds its location and type. The aggregated prediction holds its location, type, a weight representing how confident it is of its position, and a timer representing the number of frames since the last time it was matched with. Each frame object projections are stored in a queue and then run through the algorithm one by one. First, get a list of all predictions within 1.5 meters of the projection. Find the closest aggregated prediction in the list with the same type as the projection, that also has a timer greater than 0. Here, there is a split in what happens to the object projection. If no aggregated prediction was found then create a new prediction, give it the position and type of the projection, and set its weight to 1 and timer to 0. Alternatively, if a prediction was found, set the position of the prediction using the following equation:

$$NewPosition = \frac{OldPosition \times Weight + ProjectionPosition}{Weight + 1}$$

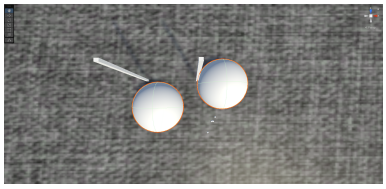
Lastly, increment the weight of the aggregated prediction and reset its timer to 0. This is the main flow of the Point Aggregation Algorithm. It achieves the goal of creating isolated predictions with positions that update as more projections are made by the system.

The algorithm runs once for each Object projection it is given, making it $\mathcal{O}(n)$. This means efficiency is not a major concern for the algorithm itself. That being said, some methods of housekeeping have been attached to the aggregated predictions. At the beginning of each frame all predictions verify that they are still actively in use. If the timer of a prediction is greater than 5 then reduce its weight by 1. If its weight is less than 1 then deactivate that object. This helps to trim down erroneous predictions from situations such as incorrect labeling or poor headset tracking. In order to create a smoother experience for the user, aggregated predictions are not rendered until their weight reaches greater than 20. When predictions are first created they jump around as they get averaged out because their weight is low. Hiding the predictions in this way helps to keep the view of the user clear until its weight is high enough to slow it down when it is averaged with new incoming projections. Once a prediction has a weight of over 30 the weight will stop changing. This allows the object to stay a permanent fixture in the scene and give it some ability to drift if the object in the real world is moved.

There were a few challenges that made developing and testing this algorithm difficult. In order to tell if the algorithm worked as intended, it needed to be tested by comparing the expected position of a prediction to its actual position. However, there were several points where problems in determining the accuracy of the algorithm arose.



(a) Scattered Projections



(b) Aggregated Predictions

Figure 7: A side by side comparison of projections and their corresponding aggregated predictions

The camera used to capture the view of the headset was not firmly attached, and if it was not delicately handled the camera could shift. the virtual position of the camera also needed to be constant every time a trial was run, or else the camera would start in the wrong position and results were skewed. Lining up the virtual environment also proved to be difficult. Another challenge of recording the positions of objects in a scene is distinguishing between multiple objects of the same type that are near each other within a single scene. Projections of these objects will match a single point aggregation within the algorithm that would trend towards the midpoint of two clusters of projections. To solve this, each point aggregation keeps track of whether or not it has been matched with each frame. Once a point aggregation has been matched it will not match with another projection that frame. This guarantees that multiple projections with the same label are not attributed to the same point aggregation.

This algorithm was the best choice for our goals and implementation of this system, but it has some limitations that may make expanding the scope of this project difficult. Currently, because of the way projections cast, this project can only be operated in a setting that has been prepared as a unity scene. The ray casts get the location of the first collision in the unity scene, meaning that with no walls desks or floor the rays wont make projections. Using just a floor will give skewed data for objects that are not on the floor, which severely throws off the accuracy of the point aggregation algorithm. Another limitation of the algorithm is its inability to keep up with moving objects. As the weight of a given prediction increases, its ability to reposition worsens. At its highest weight value, new projections will only move a prediction by one thirtieth of the distance to a new projection. If the project were to be expanded to include tracking for moving objects, then the way objects are tracked would need to fundamentally change. With that being said, the goal of this system is to prove that Language learning can be enhanced through the use of augmented reality, and these limitations do not hinder that goal.

3.7 Interactive Re-training of Image Classification (Martin)

For our system, Yolov8's pretrained model, yolov8n.pt, was used and was retrained to see how well it could be fine tuned and also see the effects of it. The original goal was to see if it was possible to resume training on a pretrained model with new data and avoid retraining on the entire old dataset which would be very time costly. A python script was written to train the model for ten epochs using two different datasets.

The first dataset, the subject is a phone and seventy images. The second dataset, the subject is a bottle and one hundred images. Both datasets are custom and labeled using labelImg.

In order to measure how much the model was forgetting a threshold confidence level of seventy percent was used. When analyzing the data, for the first dataset, the model forgot slowly over the ten epochs and showed improvement in certain areas including the subject it was being trained on. For the second dataset, the model reached catastrophic forgetting after the fourth epoch. Both datasets were custom and varied in dimensions from another. Also the subjects in training had different levels of magnification in each dataset. More testing and training needs to be done in order to learn how to deal with catastrophic forgetting. In the future, training will be done using the COCO dataset as the pretrained model was trained on that dataset and will be filtered for only relevant items. This will ensure the model is being trained on proper and consistent variation and is also known as incremental learning where the model is trained on new and old data. Unless custom fine tune is needed then a custom dataset will be made and train the model using different training techniques including incremental learning.

4 Experimental Design and Results(Tom) P5.5-P6

In the future, this section will contain experiment results. As shown in Figure **??**, data will be collected in the real time, recorded, and compared with different approaches.

5 Discussion and Future Work

6 Conclusion

References

[1] X. Huang, D. Zou, G. Cheng, and H. Xie. A systematic review of ar and vr enhanced language learning. *Sustainability*, 13(9):4639, 2021.

[2] A. Ibrahim, B. Huynh, J. Downey, T. Höllerer, D. Chun, and J. O'donovan. Arbis pictus: A study of vocabulary learning with augmented reality. *IEEE transactions on visualization and computer graphics*, 24(11):2867–2874, 2018.

[3] K. Kanayama and K. Kasahara. The indirect effects of testing: Can poor performance in a vocabulary quiz lead to long-term l2 vocabulary retention. *Vocabulary Learning and Instruction*, 7(1):1–13, 2018.

[4] A. Parmaxi and A. A. Demetriou. Augmented reality in language learning: A state-of-the-art review of 2014–2019. *Journal of Computer Assisted Learning*, 36(6):861–875, 2020.

[5] B. Perry. Gamifying french language learning: A case study examining a quest-based, augmented reality mobile learning-tool. *Procedia-Social and Behavioral Sciences*, 174:2308–2315, 2015.

[6] C. A. Rowland. The effect of testing versus restudy on retention: a meta-analytic review of the testing effect. *Psychological bulletin*, 140(6):1432, 2014.

[7] T. P. van Doormaal, J. A. van Doormaal, and T. Mensink. Clinical accuracy of holographic navigation using point-based registration on augmented-reality glasses. *Operative Neurosurgery*, 17(6):588, 2019.

[8] Q. Wu, S. Wang, J. Cao, B. He, C. Yu, and J. Zheng. Object recognition-based second language learning educational robot system for chinese preschool children. *IEEE Access*, 7:7301–7312, 2019.